Deep Learning for NLP

(Natural Language Processing)

Visualisations taken from Lena Voita's NLP Course For You

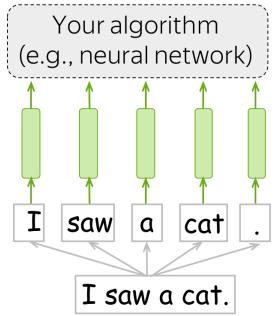
Part 1:
Word Embeddings

Word Embeddings

To apply machine learning models, we need numerical representation of data.

How to obtain representation for text?

- **Step 1**: split text into ***tokens***
- ***Step 2***: replace tokens with their ***embeddings*** (numerical vectors of features),extracted from some ***look-up table***

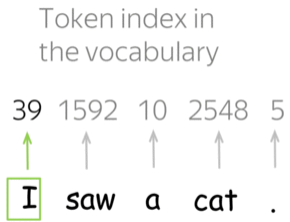


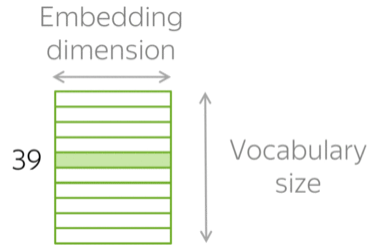
Any algorithm for solving a task

Word representation - vector
(input for your model/algorithm)

Sequence of tokens

Text (your input)

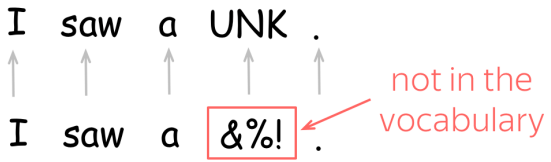




Tokenization

- Vocabulary of tokens is **fixed** in advance
- Can contain symbols, words, combinations of the sybmols/words (***n-grams***)
- Early methods also included pre-tokenization (*normalization*) step:
  - Converting to lower-case
  - Removing ***stop-words*** (“the”, “a”, “an”, “in”,...) and punctuation
  - *Stemming* - remove prefix and suffix
  - Lemmatization - turn to a correct initial form of word

- If we meet unknown token during inference - use universal token **[UNK]**



Word Embeddings

A *word* is known by the company it keeps

- J. R. Firth

Options:

- One-hot encodings
  - ✗ Vector size is too large
  - ✗ Vectors know nothing about meaning

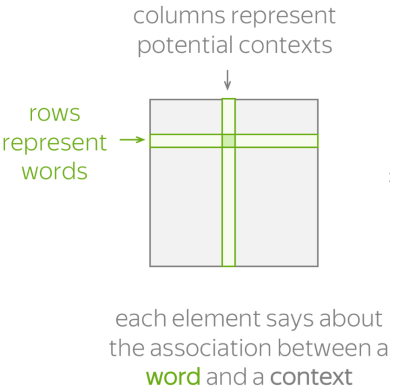
How to add context and semantic information of words to their embeddings?

Words which frequently appear in similar contexts have similar meaning.

Idea: use information about context to create word embedding

Naive solution: count-based methods

- TF-IDF - Term Frequency-Inverse Document Frequency
- Context: can be a document or a fixed window of size L
- Can apply SVD to reduce size
  - ✗ Usually computationally complex



Context:

- document d (from a collection D)

Matrix element:

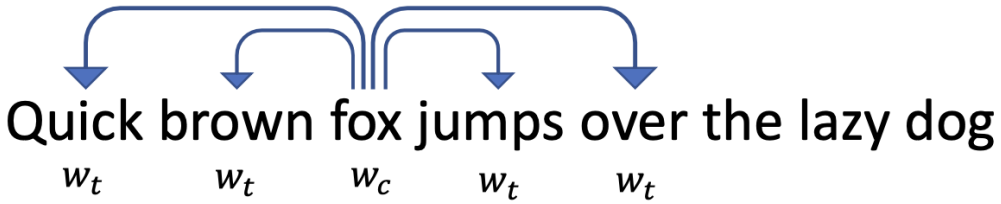
- $\text{tf-idf}(w, d, D) = \text{tf}(w, d) \cdot \text{idf}(w, D)$

$\text{N}(w, d)$ — term frequency

$\log \frac{|D|}{|\{d \in D: w \in d\}|}$ — inverse document frequency

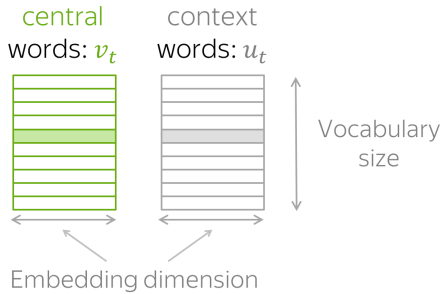
word2vec (Google 2013)

- **Learn** word vectors by teaching them to **predict contexts**
- **How**: maximize co-occurrence probability for co-occurring words



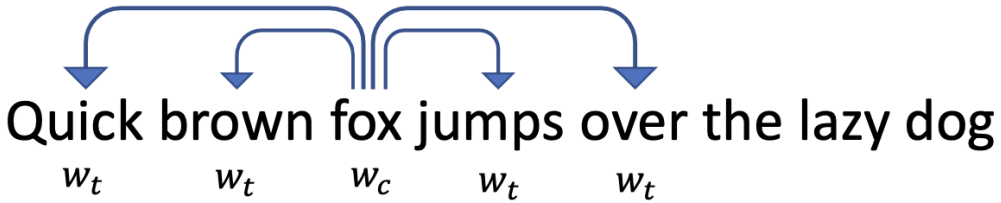
For each word w we will have two vectors:

- w_c when it is a central word
- w_t when it is a context word



word2vec (Google 2013)

- **Learn** word vectors by teaching them to **predict contexts**
- **How**: maximize co-occurrence probability for co-occurring words

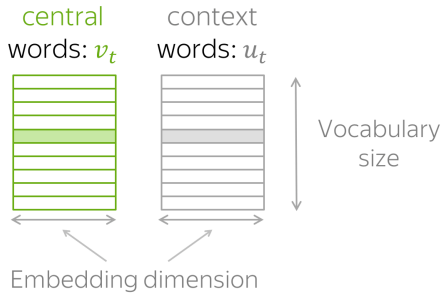


$$P(w_t \mid w_c) = \frac{exp\left(f(w_t, w_c)\right)}{\sum_{w_i \in Dict} exp\left(f(w_i, w_c)\right)}$$

Simplest approach: $f(w_t, w_c) = w_t^T \cdot w_c$ (larger dot product - larger similarity)

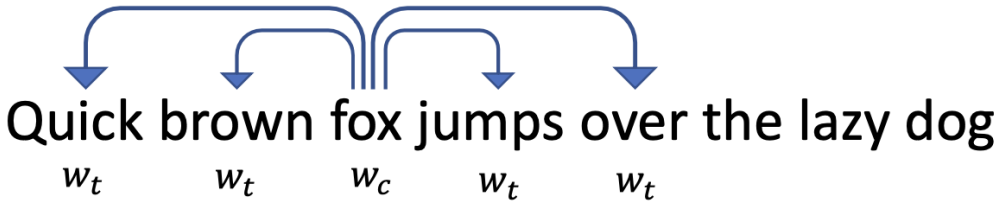
For each word w we will have two vectors:

- w_c when it is a central word
- w_t when it is a context word



word2vec (Google 2013)

- **Learn** word vectors by teaching them to **predict contexts**
- **How**: maximize co-occurrence probability for co-occurring words



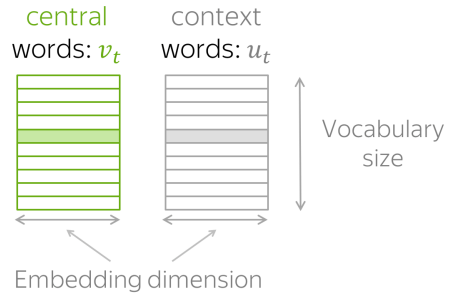
$$P(w_t \mid w_c) = \frac{exp\left(f(w_t, w_c)\right)}{\sum_{w_i \in Dict} exp\left(f(w_i, w_c)\right)}$$

Simplest approach: $f(w_t, w_c) = w_t^T \cdot w_c$ (larger dot product - larger similarity)

$$\mathcal{L} = \frac{1}{T}\sum_t L_t, \text{ where } L_t = -logP(w_t \mid w_c) = -f(w_t, w_c) + log\left(\sum_{w_i \in Dict} exp(\left(f(w_i, w_c)\right)\right)$$

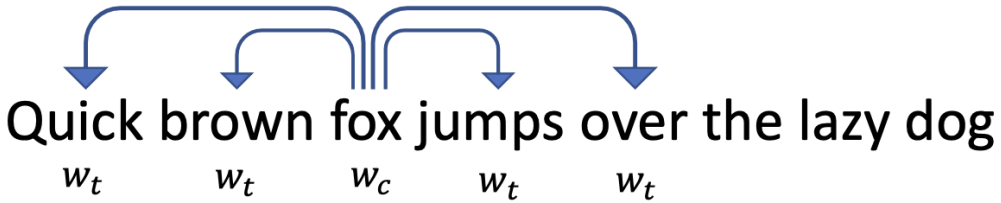
For each word w we will have two vectors:

- w_c when it is a central word
- w_t when it is a context word



word2vec (Google 2013)

- **Learn** word vectors by teaching them to **predict contexts**
- **How**: maximize co-occurrence probability for co-occurring words



$$P(w_t \mid w_c) = \frac{exp\left(f(w_t, w_c)\right)}{\sum_{w_i \in Dict} exp\left(f(w_i, w_c)\right)}$$

Simplest approach: $f(w_t, w_c) = w_t^T \cdot w_c$ (larger dot product - larger similarity)

$$\mathcal{L} = \frac{1}{T}\sum_t L_t, \text{ where } L_t = -logP(w_t \mid w_c) = -f(w_t, w_c) + log\left(\sum_{w_i \in Dict} exp(\left(f(w_i, w_c)\right)\right)$$

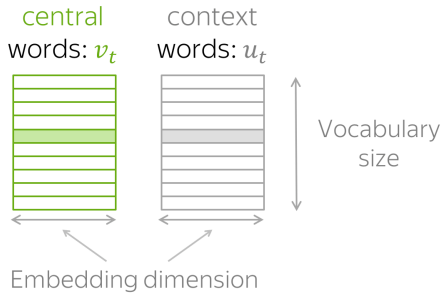
Increase similarity of w_t and w_c

Decrease similarity of w_i and w_c

train via gradient descent

For each word w we will have two vectors:

- w_c when it is a central word
- w_t when it is a context word

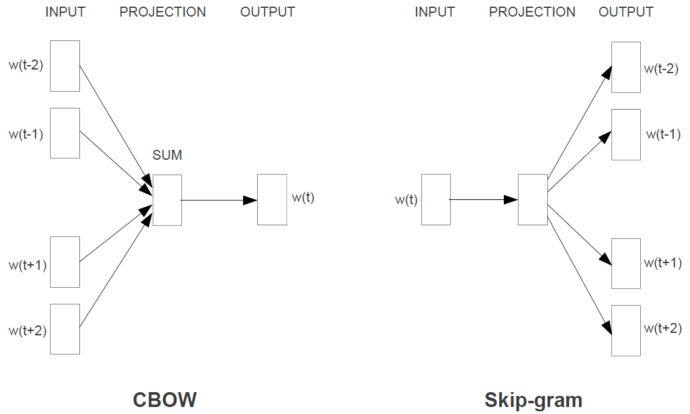


word2vec: modifications

- **Negative Sampling**

$$P(w_t \mid w_c) = \frac{exp\left(f(w_t, w_c)\right)}{\sum_{w_i \in S} exp\left(f(w_i, w_c)\right)},\ S \subset Dict, |S| = k$$

- **Skip-Gram**
- **CBOW** (Continuous *Bag of Words*)
- **Fasttext**: Adding subwords to the dictionary
  - Better understanding of morphology
  - Representations of unknown words
  - Handling misspellings



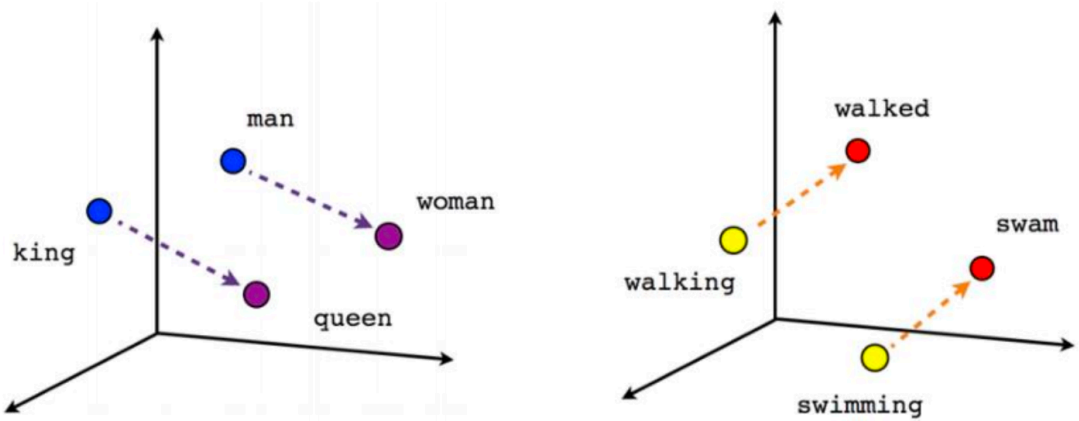
grey: <grey>+<gr+gre+rey+ey>

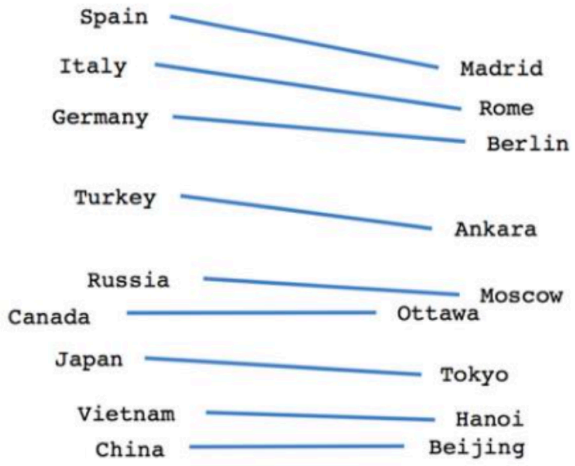
start and end characters

Most common: Skip-Gram + Negative sampling
Other popular: GloVe

Word Embeddings

Linear Structure





semantic: $v(\text{king}) - v(\text{man}) + v(\text{woman}) \approx v(\text{queen})$

syntactic: $v(\text{kings}) - v(\text{king}) + v(\text{queen}) \approx v(\text{queens})$

Word Analogy Benchmarks

Analogy: a is to a* as b is to __
Task: v(a) - v(a*) + v(b) = ?

Relationship	Example 1	Example 2	Example 3
France - Paris	Italy: Rome	Japan: Tokyo	Florida: Tallahassee
big - bigger	small: larger	cold: colder	quick: quicker
Miami - Florida	Baltimore: Maryland	Dallas: Texas	Kona: Hawaii
Einstein - scientist	Messi: midfielder	Mozart: violinist	Picasso: painter
Sarkozy - France	Berlusconi: Italy	Merkel: Germany	Koizumi: Japan
copper - Cu	zinc: Zn	gold: Au	uranium: plutonium
Berlusconi - Silvio	Sarkozy: Nicolas	Putin: Medvedev	Obama: Barack
Microsoft - Windows	Google: Android	IBM: Linux	Apple: iPhone
Microsoft - Ballmer	Google: Yahoo	IBM: McNealy	Apple: Jobs
Japan - sushi	Germany: bratwurst	France: tapas	USA: pizza

Similarities across languages

Ingredients:

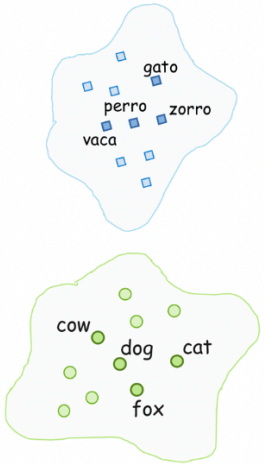
- corpus in one language (e.g., English)
- corpus in another language (e.g., Spanish)

- very small dictionary

cat ↔ gato
cow ↔ vaca
dog ↔ perro
fox ↔ zorro
…

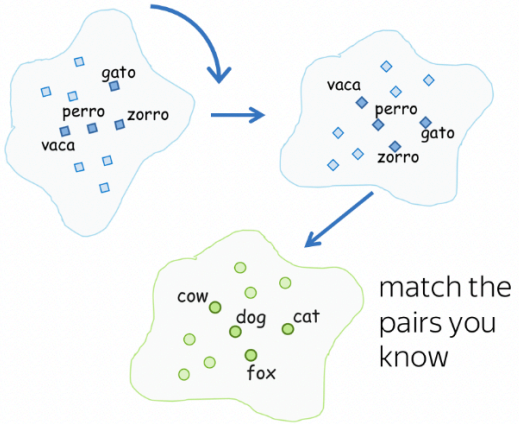
Step 1:

- train embeddings for each language



Step 2:

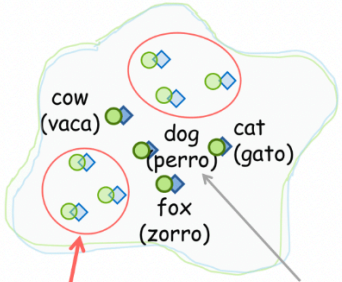
- linearly map one embeddings to the other to match words from the dictionary



match the pairs you know

Step 3:

- after matching the two spaces, get new pairs from the new matches



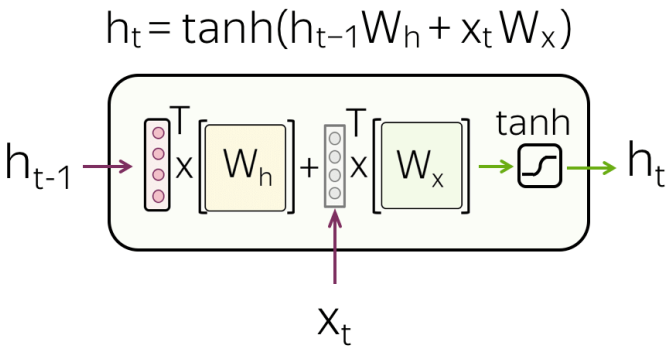
old translations – the ones you knew

new translations – you learned them!

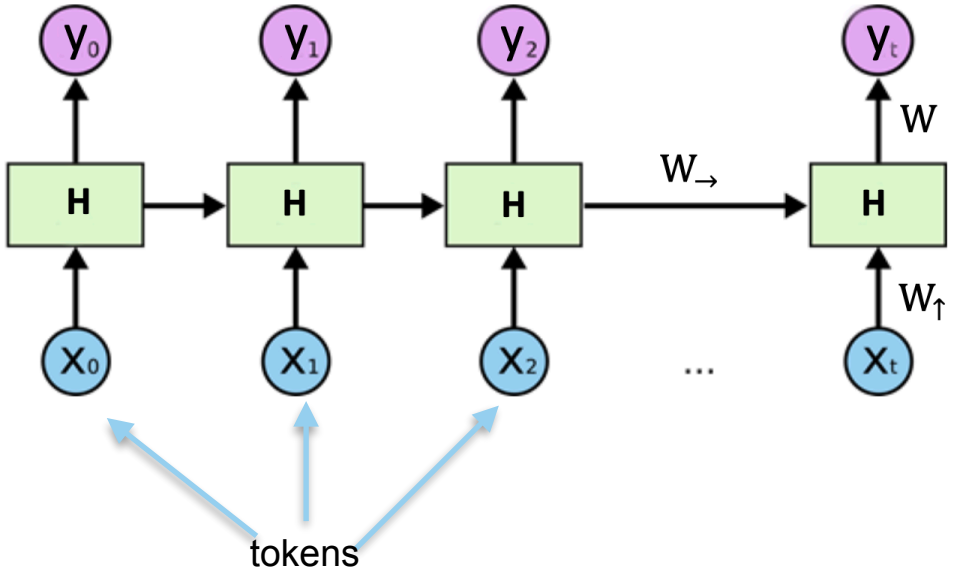
Part 2:
Deep Learning for NLP

Recurrent Neural Network

RNN Cell

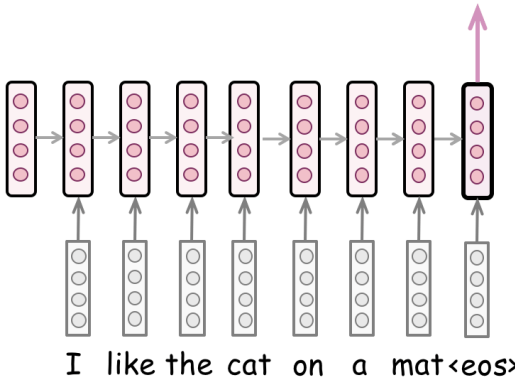


RNN for Text processing

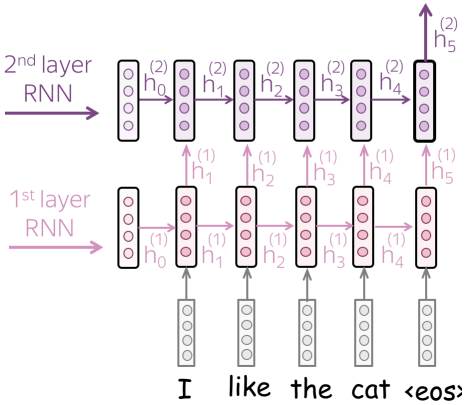


RNN for Text Classification

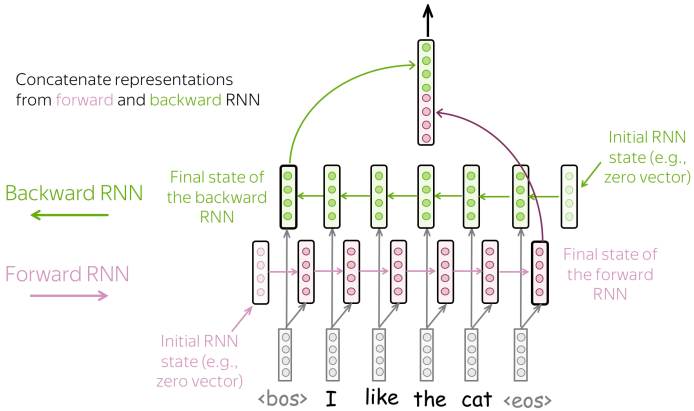
Simple: read a text, take final state



Multiple layers:
feed the states from one RNN to next one



Bidirectional:
use final states from forward and backward RNNs



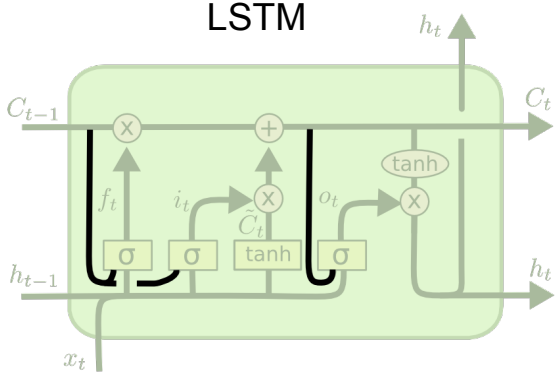
RNN Extensions

Problems of RNNs

- Vanishing and exploding gradients
- Forgetting earlier tokens ⟶ struggle to handle *long-term dependencies*

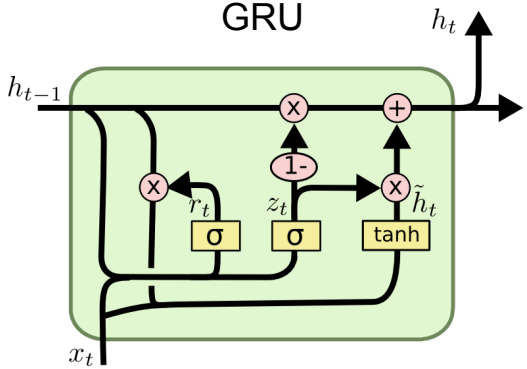
Solutions:

LSTM



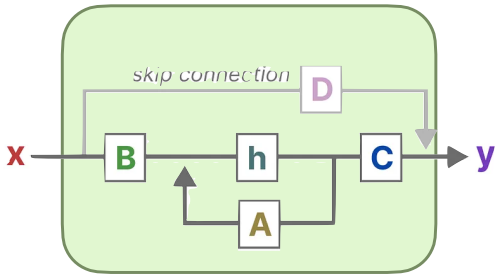
$$\textbf{forget gate} \quad f_t = \sigma(W_f \cdot [y_{t-1}, x_t] + b_f)$$
$$\textbf{input gate} \quad i_t = \sigma(W_i \cdot [y_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = tanh(W_c \cdot [y_{t-1}, x_t] + b_C)$$
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$
$$\textbf{output gate} \quad o_t = \sigma(W_o \cdot [y_{t-1}, x_t] + b_o)$$
$$y_t = o_t * \tanh(C_t)$$

GRU



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$
$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$
$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$
$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

SSM



State equation:
$h_k = Ah_{k-1} + Bx_k$

Output equation:
$y_k = Ch_k$